

Predicting JOSAA College Admissions Using an Ensemble of Year-Trend and Round-Progression Models

From Automated Data Collection to Multi-Round Forecasting

Harith Yerragolam

harith.yerragolam@research.iiit.ac.in

April 2026

Abstract

Every year, hundreds of thousands of students who qualify the Joint Entrance Examination (JEE) must navigate the Joint Seat Allocation Authority (JoSAA) counselling process to determine which college and programme they are eligible for. The process spans up to six sequential allocation rounds, and the closing rank for any given seat changes each round and each year, making manual prediction error-prone and time-consuming. This paper documents the complete design and implementation of an end-to-end system that (1) automatically scrapes the official JoSAA historical rank archive, (2) models each unique seat slot (institute $\times$ programme $\times$ quota $\times$ seat type $\times$ gender) as a two-dimensional time series over years and rounds, and (3) predicts the full R1-R6 closing-rank trajectory for the upcoming counselling year using an ensemble that combines a per-round year-trend signal with a learned round-progression ratio. The system is subsequently extended to the Central Seat Allocation Board (CSAB) supplementary counselling round, which fills leftover seats after JoSAA closes; we document the additional data challenges and show that CSAB predictions carry substantially higher uncertainty (overall MAE $\approx 49,900$ with GP RBF, reduced to $\approx 42,300$ with the GP-MLP ensemble on the 2025 test year and $\approx 37,500$ on the 2024 test year, versus $\approx 2,858$ for JoSAA with the GP-MLP ensemble on the 2025 test year), reflecting the inherently chaotic nature of residual seat allocation. A systematic comparison of nine year-trend models reveals that JoSAA closing ranks are mean-reverting rather than trending, which explains why the historical median outperforms all linear extrapolation methods by 23% on average. A multi-year validation (2022-2025) shows that a GP-MLP ensemble with a tuned architecture ((512, 256, 128), dropout 0.15, $\alpha=0.5$) achieves the best average MAE of 2,608, outperforming GP RBF (2,667) and SVR RBF (2,742), and is deployed as the default for both JoSAA and CSAB. We describe the engineering challenges encountered during data collection, motivate every architectural decision, and provide a backtesting framework to evaluate prediction accuracy.

1 Introduction and Motivation

1.1 The JoSAA Counselling Process

JoSAA (Joint Seat Allocation Authority) coordinates admissions to ~120 premier engineering institutions in India: the Indian Institutes of Technology (IITs), National Institutes of Technology (NITs), Indian Institutes of Information Technology (IIITs), and other Government-Funded Technical Institutions (GFTIs). Each year, qualified candidates are allocated seats through a multi-round process. The two underlying examinations are:

- **JEE Advanced** - used exclusively for IIT admissions; ranks students on a Common Rank List (CRL) as well as category-specific lists.
- **JEE Mains** - used for NIT, IIIT, and GFTI admissions.

A single counselling year comprises up to six rounds. In each round, a candidate is offered a seat at the best institute-programme combination available given their rank. They may accept (freeze), float (keep the seat and participate in further rounds hoping for a better option), or slide (accept a different programme at the same institute). After all rounds, the final allocated seat is binding. The underlying allocation algorithm is a Multi-Round Multi-Run variant of Deferred Acceptance (Gale-Shapley), extended to handle multiple merit lists, reservation categories, and supernumerary seats; its design and large-scale implementation are described in [1, 2].

The *closing rank* of a programme in a given round is the rank of the last student allocated to that programme in that round. It is the primary signal a prospective student uses to judge eligibility.

1.2 The Problem with Manual Prediction

Prior to this system, students navigated the JoSAA historical archive manually, selecting year, round, and filter parameters one at a time and copying the resulting HTML tables into spreadsheets - up to 54 form-submission cycles for a complete dataset. The resulting analysis was static (no future-year extrapolation), error-prone (manual transcription), and incomplete (without all rounds the within-year trajectory was invisible).

1.3 Contributions

This work makes the following contributions:

1. **Automated scrapers** for the JoSAA historical archive, the CSAB supplementary archive, and the JoSAA seat matrix, each handling the site’s ASP.NET state management and jQuery Chosen UI widgets with resume-safe crash recovery.
2. **Datasets:** 549,726 JoSAA rows (2016-2026; all rounds for 2016-2025, Rounds 1-2 for 2026), 47,066 CSAB rows (2021-2025), and 13,343 seat-matrix rows for the current year, all in a clean structured CSV format.
3. **Two-dimensional SlotModel ensemble** combining a per-round year-trend signal with learned round-progression ratios to output a full R_1 - $R_{\max}$ closing-rank trajectory; non-parametric per-slot prediction intervals and calibrated safe/match/reach eligibility categories.

4. **Systematic trend-model comparison** across nine variants (OLS, Theil-Sen, Weighted OLS, Ridge, SVR Linear, SVR RBF, AR(1), AR(p), GP RBF) and four held-out years (2022-2025); establishes the mean-reversion finding; GP RBF achieves the best pure trend-model MAE (**2,667**, -18% vs Median).
5. **Global MLP with learned categorical embeddings** trained jointly on all 549k rows; achieves average MAE **2,676**, handles cold-start slots, and forms the MLP component of the deployed GP-MLP ensemble.
6. **GP-MLP ensemble** deployed as the default for both sources: **JoSAA** uses tuned arch (512, 256, 128), dropout 0.15, $\alpha=0.5$ (average MAE **2,608**, -0.9% vs GP RBF); **CSAB** uses the default arch with LOO-calibrated $\alpha=0.7$ (overall MAE **42,318**, -15.1% vs GP RBF alone), with source-aware scrapers, quota normalisation, and widened eligibility thresholds.
7. **Streamlit web application** with searchable programme/institute filters, interactive Plotly closing-rank trajectory plot, category-wise CSV export, and seat-matrix integration; a CLI with `train/backtest/tune/predict` subcommands.

1.4 Related Work

JoSAA seat-allocation algorithm. The algorithmic machinery behind JoSAA has been studied formally. Baswana et al. [1] describe the Multi-Round Multi-Run Deferred Acceptance algorithm introduced in 2015 that centralises allocation across ≈ 100 institutes and $\approx 39,000$ seats, handling multiple merit lists, reservation rules, rank escalations, and supernumerary quotas. Kanoria et al. [2] present the broader design rationale and real-world impact of centralised admissions in India. Singh and Saxena [3] study the theoretical seat-allocation problem with multiple merit lists and propose an improved, transparency-oriented algorithm. These works focus on *allocation mechanism design*; none address the prediction of future closing ranks from historical data.

ML-based college admission prediction. Several recent papers apply machine learning to predict engineering college admissions in India. Salgaonkar et al. [4] propose a recommendation system for MHT-CET (Maharashtra state exam) that uses regression models on percentile, rank, and previous-year cutoffs to generate preference lists. JETIR (2025) [5] develops a web application using classification algorithms for predicting admission outcomes for JEE-ranked students. An ensemble approach stacking decision trees, k-NN, Naïve Bayes, and one-vs-rest SVM is presented in [6]. These systems typically frame admission prediction as a *feasibility classification* (admitted or not), and rely on matching a student’s rank against historical cutoff ranges with a fixed buffer. By contrast, this work treats each institute-programme-category combination as a time series and directly forecasts the closing rank for the upcoming year across all rounds, enabling multi-round trajectory prediction rather than single-round eligibility estimation.

Time-series rank processes. To the best of our knowledge, no prior work formally models JoSAA closing ranks as a mean-reverting time series, nor demonstrates empirically that linear trend extrapolation is inferior to the historical median in this domain. The

mean-reversion finding (Section 7.1) and the SVR-RBF result (Section 7.2) are, to our knowledge, new contributions.

2 Data Collection

2.1 Target Website and Scraping Approach

The JoSAA archive [7, 8] is an ASP.NET WebForms application whose dropdowns use the *jQuery Chosen* plugin, which hides native `<select>` elements behind custom `<div>` widgets. Every dropdown change triggers a *postback* that regenerates the page’s hidden `__VIEWSTATE` token, making static HTTP requests impossible. We use **Microsoft Playwright** (Python bindings), which drives a real Chromium instance so ViewState is managed by the browser automatically. Chosen dropdowns are operated by injecting JavaScript via `page.evaluate()` to set the native select value and fire `trigger('change')`, followed by `wait_for_load_state("networkidle")` before proceeding.

2.2 Scraping Architecture

The outer loop iterates over years; for each year, an inner loop iterates over rounds. Completed (year, round) pairs are read from the existing CSV on startup and skipped, giving resume safety after crashes. Each new pair sets all filter dropdowns to ALL, submits, waits for `networkidle`, extracts the largest table from the DOM, and appends rows with an immediate flush. Each dropdown selection waits for the cascaded option list to populate (polled up to 8 seconds) rather than using a fixed sleep.

2.3 Scraping the 2025 Current-Year Data

Current-year data is served from a separate live page with no Year dropdown. A dedicated scraper handles it using the same Playwright infrastructure, noting that the 2025 page pre-populates all dropdowns on load so no per-dropdown postback wait is needed.¹ The complete collection run took approximately **4 hours**; mid-run crashes required only a restart, not a full re-scrape.

2.4 CSAB Extension

The Central Seat Allocation Board (CSAB) runs a supplementary round after JoSAA closes [9, 10]. Its archive page has a structurally different form (no Seat Type or Quota dropdowns; 2-3 rounds per year rather than 6) and uses full quota names (“All India”, “Home State”) in the current-year page versus abbreviations (“AI”, “HS”) in historical data - a mismatch resolved by a normalisation map in the loader. Dedicated scrapers follow the same resume-safe Playwright architecture and write to a separate CSV.

¹A case-sensitivity quirk in the ASP.NET-generated element IDs (`ddlInstype`, not `ddlInsType`) caused selector timeouts and was fixed by inspecting the actual IDs at runtime.

2.5 Seat Matrix Scraper

In addition to historical closing ranks, the JoSAA portal publishes a *seat matrix* for the current year: the number of available seats broken down by institute, programme, quota, seat type, and gender. This data is available at `seatmatrixinfo.aspx` [11] and is useful for contextualising predictions - a programme with few seats and a closing rank close to a student’s rank carries more risk than one with many seats.

Simpler scraping challenge. Unlike the closing-rank archive, the seat matrix page does *not* use jQuery Chosen widgets. All dropdowns default to ALL, so a single button click triggers a standard ASP.NET postback that returns all 4,052 data rows. Each programme $\times$ quota combination spans three HTML rows (Gender-Neutral, a rowspan continuation row, and Female-only); the scraper distinguishes them by cell count and maps raw column names to pipeline codes (e.g. GEN-EWS $\rightarrow$ EWS).

Quota aggregation. The seat matrix uses state-specific NIT quota names (e.g. “PUNJAB”, “Other than PUNJAB”); these are normalised to HS/OS before aggregation to match the pipeline’s quota codes. Special codes AI, GO, JK, LA are kept as-is. The 2025 scrape produced **13,343 rows** across all seat configurations.

2.6 Dataset Description

Table 1: Dataset statistics after scraping all available years and rounds.

Property	JoSAA	CSAB	Seat Matrix
Years available	2016-2026 (11 years; Rounds 1-2 for 2026)	2021-2025 (5 years)	2025 (current year)
Rows	549,726	47,066	13,343
Unique slot models	17,624	9,171	-
Institute types	IIT, NIT, IIIT, GFTI	NIT, IIIT, GFTI	NIT, IIIT, GFTI
Quota values	AI, HS, OS, GO, JK, LA	same (normalised)	AI, HS, OS, GO, JK, LA
Seat types	OPEN, EWS, OBC-NCL, SC, ST ...	same	same
Gender values	Gender-Neutral, Female-only	Gender-Neutral, Female-only	Gender-Neutral, Female-only
Scraping time	$\approx$ 4 hours	$\approx$ 30 minutes	<1 minute

Table 2: CSV column schema.

Column	Type	Description
Year	int	Academic year
Round	int	Counselling round (1-6)
Institute	string	Full institute name
Academic Program Name	string	Degree + duration + branch
Quota	categorical	Seat quota (AI / HS / OS ...)
Seat Type	categorical	Reservation category
Gender	categorical	Gender-Neutral / Female-only
Opening Rank	int	Best rank allocated in this slot
Closing Rank	int	Worst rank allocated in this slot

3 Problem Formulation

3.1 Slot Definition

We define a *slot* as the unique combination:

$$s = (\text{Institute}, \text{Programme}, \text{Quota}, \text{Seat Type}, \text{Gender}, \text{Exam Type})$$

where **Exam Type** $\in \{\text{advanced}, \text{mains}\}$ is inferred from the institute name (institutes containing “Indian Institute of Technology” map to *advanced*; all others map to *mains*). This is critical because JEE Advanced and JEE Mains produce entirely independent rank sequences; conflating them would be meaningless.

For each slot s and year y , round r , the dataset contains:

$$C_{s,y,r} = \text{closing rank of slot } s \text{ in year } y, \text{ round } r$$

3.2 Prediction Task

Given a student profile $p = (\text{rank}, \text{exam_type}, \text{quota}, \text{seat_type}, \text{gender})$ and a target year Y_{pred} , return a ranked list of eligible slots together with predicted closing ranks for every round:

$$\hat{C}_{s,Y_{\text{pred}},r} \quad \forall r \in \{1, 2, 3, 4, 5, 6\}$$

A slot s is *eligible* for a student with rank k if:

$$k \leq \hat{C}_{s,Y_{\text{pred}},R_{\text{max}}(s)}$$

where $R_{\text{max}}(s)$ is the last round for slot s . Predicting all rounds rather than only the final one lets a student see whether they would have received an offer in R1 and how the cutoff is expected to move - a within-year trajectory not provided by any existing tool.

4 Methodology

4.1 Overview

For each slot s we fit a **SlotModel** that encodes two complementary signals extracted from the historical matrix $\{C_{s,y,r}\}$:

1. **Year-trend signal:** how the closing rank for each round r has evolved across years.
2. **Round-progression signal:** the typical shape of the $R1 \rightarrow R2 \rightarrow \dots \rightarrow R_{\max}$ trajectory within a single year.

The final prediction is a weighted ensemble of both signals.

4.2 Year-Trend Signal

For each slot s and round r , we fit an ordinary least-squares linear regression:

$$\hat{C}_{s,r}(y) = \alpha_{s,r} \cdot y + \beta_{s,r}$$

using all years in which round r was observed for slot s .

Design choice: linear vs. non-linear regression. Closing ranks exhibit broadly monotonic trends (increasing demand for popular programmes drives ranks down; declining interest does the reverse). A linear model captures this trend without overfitting on the 5-9 available data points per slot. Non-linear models (e.g. polynomial, neural networks) were considered but rejected at this stage due to the small per-slot sample size.

Minimum data requirement. Slots with fewer than `MIN_YEARS_FOR_TREND`= 2 observations for a given round do not receive a trend model; they fall back to the historical median closing rank for that round.

4.3 Round-Progression Signal

Closing ranks are not independent across rounds: floating students vacate seats in later rounds, causing popular programmes to tighten and less popular ones to loosen. These patterns are captured by the *round-progression ratio*. For slot s and year y , define:

$$\rho_{s,r,y} = \frac{C_{s,y,r}}{C_{s,y,R_{\max}(y)}}$$

i.e. the closing rank of round r relative to the final round's closing rank in the same year. We then average across years to obtain a stable per-round ratio:

$$\bar{\rho}_{s,r} = \frac{1}{|\mathcal{Y}_{s,r}|} \sum_{y \in \mathcal{Y}_{s,r}} \rho_{s,r,y}$$

where $\mathcal{Y}_{s,r}$ is the set of years in which both round r and the final round are observed for slot s .

Given a predicted final-round closing rank $\hat{C}_{s,Y_{\text{pred}},R_{\max}}$, the round-ratio prediction for round r is:

$$\hat{C}_{s,Y_{\text{pred}},r}^{\text{ratio}} = \hat{C}_{s,Y_{\text{pred}},R_{\max}} \cdot \bar{\rho}_{s,r}$$

4.4 Ensemble Combination

Let $\hat{C}_{s,r}^{\text{trend}}(Y)$ denote the direct year-trend prediction for round r in year Y , and $\hat{C}_{s,r}^{\text{ratio}}(Y)$ the ratio-scaled prediction. The ensemble prediction is:

$$\hat{C}_{s,Y,r} = w \cdot \hat{C}_{s,r}^{\text{trend}}(Y) + (1 - w) \cdot \hat{C}_{s,r}^{\text{ratio}}(Y)$$

with ensemble weight $w \in [0, 1]$ (default $w = 1.0$; see Section 6.6 for the tuning procedure).

4.5 In-Season Multi-Round Anchoring

As each round’s actual closing ranks are announced during a live counselling year, they provide ground truth that is strictly more informative than any year-trend extrapolation. We exploit this by re-centring the model’s remaining-round trajectory on a weighted aggregate of all observed actuals.

Per-round final-close estimate. Each observed round r with actual closing rank $C_{s,r}^*$ yields an independent estimate of the final-round closing rank via the stored round-progression ratio:

$$\hat{F}_s^{(r)} = \frac{C_{s,r}^*}{\bar{\rho}_{s,r}}$$

Using a single observed round and setting $r=1$ recovers the earlier single-anchor formula. Across all JoSAA slots in 2016-2025, the mean $R1/R_{\max}$ ratio is $\bar{\rho}_{s,1} \approx 0.864$ (median 0.886), confirming that Round 1 closes at $\approx 87\%$ of the final-round rank.

Weighted aggregate anchor. Later rounds have lower ratio variance (they are closer to the final round), making their estimates of F_s more reliable. We use round number as a reliability proxy and compute a weighted average:

$$\hat{F}_s = \frac{\sum_{r \in \mathcal{O}_s} r \cdot \hat{F}_s^{(r)}}{\sum_{r \in \mathcal{O}_s} r}$$

where $\mathcal{O}_s$ is the set of rounds with observed actuals for slot s . The anchored prediction for any remaining round k is then:

$$\hat{C}_{s,k}^{\text{anc}} = \hat{F}_s \cdot \bar{\rho}_{s,k}$$

With only R1 available, $\hat{F}_s = C_{s,1}^*/\bar{\rho}_{s,1}$ (100% R1). With R1 and R2 both available, R1 contributes 33% and R2 contributes 67%. With R1-R3, weights are 17%/33%/50%. Each new round automatically increases the weight on the most recent (most reliable) observation.

Prediction interval after anchoring. Only the centre of the interval shifts; the historical half-width $h_{s,r}(\alpha)$ (Section 5.3) is preserved because year-to-year ratio variability is unaffected by knowing the observed rounds:

$$\text{interval} = \left[\max(1, \hat{C}_{s,k}^{\text{anc}} - h_{s,k}(\alpha)), \hat{C}_{s,k}^{\text{anc}} + h_{s,k}(\alpha) \right]$$

Implementation. `pipeline/config.py` defines `CURRENT_ROUND_DATA: dict[int, str]`, mapping round numbers to CSV file paths. Adding a new round’s data requires a single-line config change; all downstream logic (anchoring, evaluation, UI) adapts automatically. `load_all_actuals()` reads every configured CSV into a `{round: {slot_key: closing_rank}}` dict. `predict()` accepts `actual_rounds` and calls `_anchor_with_actuals()` for any slot with at least one observed round; unmatched slots fall back to the pure model prediction. `evaluate_round(model, actuals, round_num)` compares model predictions against actuals for any round and returns per-slot errors; the Streamlit sidebar renders MAE, median AE, and a progress-bar error-distribution chart (percentage of slots within ± 500 , $\pm 1,000$, $\pm 2,000$, $\pm 5,000$ ranks) for each available round. The 2026 deployment uses Rounds 1-2 as anchors. Round 1 anchored 11,716 of 13,292 R1 slots (1,576 newly-appeared slots had no historical model and remained unanchored); Round 2 contributes 13,047 observed actuals and receives twice the weight of Round 1 ($r=2$ vs $r=1$) in the weighted aggregate formula, improving the final-round estimate for all matched slots.

Enabling `get_round_ratios()`. Anchoring requires each slot model to expose its stored $\bar{\rho}_{s,r}$ values. A `get_round_ratios()` method was added to three adapter classes: `SlotModel` (direct lookup), `_GPMLPSlotAdapter` (delegates to the per-slot GP model, falling back to ratios derived from stored round medians), and `_GlobalSlotAdapter` (reads from `slot_stats["round_ratios"]` computed during `GlobalMLPModel.fit()`).

4.6 Prediction Output and Eligibility Categories

For a student with rank k , slot s is categorised as:

$$\text{Category}(s, k) = \begin{cases} \text{safe} & \text{if } k \leq 0.8 \cdot \hat{C}_{s,Y,R_{\max}} \\ \text{match} & \text{if } 0.8 \cdot \hat{C}_{s,Y,R_{\max}} < k \leq \hat{C}_{s,Y,R_{\max}} \\ \text{reach} & \text{if } \hat{C}_{s,Y,R_{\max}} < k \leq 1.2 \cdot \hat{C}_{s,Y,R_{\max}} \\ \text{ineligible} & \text{otherwise} \end{cases}$$

The thresholds 0.8 and 1.2 are heuristic; the 20% reach buffer acknowledges that predicted closing ranks carry uncertainty and actual cutoffs shift year-to-year.

5 Software Architecture

5.1 Codebase Structure

The repository contains a `scripts/` directory holding five scraper scripts, the CLI entry point (`predict_cli.py`), and a model-comparison script; a `pipeline/` package (loader, trainer, predictor, evaluator); a Streamlit web application (`app.py`); and serialised `SlotModel` objects in `models/` as pickle files.

5.2 Training Pipeline

1. `loader.load(csv_path)`: read CSV, cast types, strip PwD rank prefixes ("P"), infer exam type.

2. Group by `SLOT_COLS`; for each group, instantiate and fit a `SlotModel`.
3. Serialise the dictionary of `SlotModel` objects with `pickle` to `models/josaa_model.pkl`.

5.3 Prediction Intervals and Calibrated Eligibility Categories

The fixed eligibility thresholds ($\text{safe} \leq 0.80\hat{C}$, $\text{reach} \leq 1.20\hat{C}$) do not account for per-slot uncertainty: a programme whose closing rank has historically varied by ± 200 positions deserves a much narrower interval than one that has swung by $\pm 8,000$. We implement a non-parametric, per-slot prediction interval that replaces the fixed thresholds.

5.3.1 Historical Absolute Deviations

During training, after fitting the year-trend and round-progression models, each `SlotModel` computes the *absolute deviation of every historical closing rank from the per-round historical median*:

$$d_{s,r,y} = |C_{s,y,r} - \tilde{C}_{s,r}|$$

where $\tilde{C}_{s,r}$ is the median of $\{C_{s,y,r}\}$ across training years. The sorted list of deviations $\{d_{s,r,y}\}$ is stored in `SlotModel.round_abs_deviations[r]`.

5.3.2 Prediction Interval

For a coverage level α (default 0.90), the half-width is the $\lceil n\alpha \rceil$ -th smallest deviation where n is the number of training years for that round:

$$h_{s,r}(\alpha) = d_{s,r}^{(\lceil n\alpha \rceil)}$$

The prediction interval for slot s , round r , year Y is then:

$$\left[\max(1, \hat{C}_{s,Y,r} - h_{s,r}(\alpha)), \hat{C}_{s,Y,r} + h_{s,r}(\alpha) \right]$$

Slots with fewer than two training years fall back to a $\pm 20\%$ band.

5.3.3 Calibrated Eligibility Categories

The student's rank k is compared against the interval at the final round $R_{\max}(s)$:

$$\text{Category}(s, k) = \begin{cases} \text{safe} & k \leq \hat{C}_{s,Y,R_{\max}} - h_{s,R_{\max}}(\alpha) \\ \text{match} & \hat{C}_{s,Y,R_{\max}} - h_{s,R_{\max}}(\alpha) < k \leq \hat{C}_{s,Y,R_{\max}} \\ \text{reach} & \hat{C}_{s,Y,R_{\max}} < k \leq \hat{C}_{s,Y,R_{\max}} + h_{s,R_{\max}}(\alpha) \end{cases}$$

This replaces the previous fixed 0.80/1.20 thresholds. Volatile slots (large historical deviations) automatically generate wider safe/reach bands; stable slots generate narrower ones. The web UI uses a fixed default of $\alpha = 0.95$; it can be adjusted via the `-coverage` flag in the CLI.

5.3.4 Limitations

The half-width $h_{s,r}(\alpha)$ is computed from the in-sample deviations of historical ranks from their median, *not* from leave-one-out prediction residuals. For the Median trend model, the two are equivalent; for the GP-MLP ensemble (the current default), in-sample fit is tighter than LOO fit, so the intervals will be slightly optimistic. A future improvement is to replace this with proper leave-one-year-out residuals. Despite this, the intervals are substantially more informative than fixed percentage thresholds because they reflect each slot's actual historical volatility. The web interface surfaces per-slot data availability to the user via a groundedness indicator (Section 5.6), making it transparent when a prediction relies on fewer than three years of history.

5.4 Inference Pipeline

1. Load the serialised model pickle and, if available, the seat matrix. An absent seat-matrix file returns an empty dict with no error.
2. Filter slots by exam type, quota, seat type, and gender.
3. For each matching slot, call `SlotModel.predict_all_rounds(year, rounds)`.
4. Compute prediction interval via `SlotModel.predict_interval` at the requested coverage level (default 0.90); categorise as safe / match / reach based on where the student's rank falls relative to the interval (Section 5.3).
5. Look up (institute, programme, quota, seat_type, gender) in the seat-matrix dict; attach the integer seat count, or "-" if the slot is absent.
6. Sort by (category, predicted final-round closing rank ascending).

5.5 CLI and Web Interface

`scripts/predict_cli.py` exposes four subcommands: `train`, `backtest`, `tune` (grid-searches ensemble weight w with optional `-save`), and `predict`. All accept `-source` {josaa, csab}, `-trend-model`, and `-coverage`.

5.6 Web User Interface

A **Streamlit** web application (`app.py`) exposes the same prediction engine through a point-and-click workflow. Sidebar widgets capture the student profile (source, exam type, rank, quota, seat type, gender). Results are grouped into colour-coded safe/match/reach tables with searchable programme and institute filters, training-year counts, seat-matrix capacity, and CSV export. A second tab renders a **Plotly** interactive trajectory chart showing the predicted R_1 - $R_{\max}$ closing-rank curves for user-selected colleges, with an inverted Y-axis (lower = more accessible) and the student's rank overlaid as a reference line. All model objects are cached with `@st.cache_resource` so re-queries within a session do not reload the pickle.

A per-slot *groundedness badge* is displayed on the slot history page, colour-coded by the number of years of historical closing-rank data available for that exact slot: **strong** (≥ 5 years, green), **moderate** (≥ 3 years, orange), or **weak** (< 3 years, red). This gives

users an at-a-glance signal of how much historical evidence backs each prediction before they act on it, and is directly linked to the fallback logic in Section 5.3: slots flagged as weak are precisely those for which the prediction interval falls back to a $\pm 20\%$ band rather than a data-driven half-width.

6 Evaluation

6.1 Backtesting Protocol

We evaluate in a *leave-one-year-out* fashion: train on years $\{2016, \dots, Y-1\}$ and predict year Y . For each slot present in the test year, we compare the predicted closing rank $\hat{C}_{s,Y,r}$ against the actual $C_{s,Y,r}$ using Mean Absolute Error (MAE):

$$\text{MAE}_r = \frac{1}{|\mathcal{S}_r|} \sum_{s \in \mathcal{S}_r} |\hat{C}_{s,Y,r} - C_{s,Y,r}|$$

where $\mathcal{S}_r$ is the set of slots for which round r exists in the test year.

6.2 Data Quality Fix: Missing Gender Column (2016-2017)

During the first backtest run, the training set unexpectedly covered only 2018-2023 despite 2016 and 2017 being present in the CSV (25,601 and 30,177 rows respectively). Tracing through the loader step by step revealed that *all* 2016/2017 rows were dropped by the initial `dropna` call because the `Gender` column was entirely `NaN` for those years.

The root cause is a change in the JoSAA seat matrix format: the *Female-only (including Supernumerary)* seat category was introduced in 2018. Before that, all seats were implicitly gender-neutral and the data table had no `Gender` column. The scraper copied the table as-is, so 2016/2017 rows have no `Gender` value.

The fix in `loader.py` fills missing `Gender` values with "Gender-Neutral" (or adds the column entirely if absent). This recovers all 55,778 rows from 2016-2017 and extends the effective training window from 7 to 9 years (2016-2024 for the 2024 backtest).

6.3 Training

Training was run on the complete dataset (after the gender-column fix recovered 2016-2017 and the 2026 R1-R2 rows were appended) using `python scripts/predict_cli.py train`.

- **540,675** rows (after loader cleaning of the 549,726-row raw CSV) across **11 years** (2016-2026; Rounds 1-2 for 2026) and up to **7** rounds per year.
- **14,474** GP slots and **17,624** total slot adapters trained and serialised to `models/josaa_model.pkl`.
- Training completed in approximately **5 minutes** on an RTX 5060 Laptop GPU (CUDA 12.8, PyTorch 2.11).

6.4 JoSAA Results

Table 3: Per-round MAE with the **median** trend model on held-out year 2025, trained on 2016-2024 (9 years).

Round	N Slots	MAE (rank positions)
R1	11,156	3,323
R2	10,996	3,730
R3	10,949	3,776
R4	10,944	3,793
R5	10,940	3,820
R6	10,924	4,136
Overall	65,909	3,761

Progressive improvements to overall MAE.

1. OLS with 2018-2023 data (missing gender fix): **5,911.5**
2. OLS with 2016-2024 data (after gender fix): **5,027.7** (−15%)
3. Median with 2016-2024 data: **3,761.3** (−36% vs. step 1, −25% vs. step 2)

The largest single gain came from switching to the median trend model, which revealed that closing ranks are mean-reverting rather than linearly trending (see Section 7.1).

Interpreting the MAE. The overall MAE must be read in context of the rank scale: JEE Mains closing ranks for GFTIs can exceed 500,000, so a $\sim 3,000$ -rank error is $< 1\%$. Stratified results (Section 6.5) use SVR RBF as a representative single-component baseline; the modest R1-to-R6 MAE increase is consistent with later rounds being more influenced by floating/sliding behaviour.

6.5 Stratified Evaluation by Exam Type and Institute Tier

The tables below use SVR RBF ($w = 1.0$, 2025 test year, overall MAE 2,834) as a single-component baseline for stratification; the deployed GP-MLP ensemble achieves 2,608 on average and the tier-level patterns are qualitatively identical. JEE Advanced slots (IITs; closing ranks ~ 100 -20,000) and JEE Mains slots (NITs, IIITs, GFTIs; closing ranks $\sim 1,000$ -500,000) are broken down by exam type and institute tier in Table 4 and Table 5 respectively.

Table 4: MAE stratified by exam type, test year 2025 (SVR RBF, $w = 1.0$, JoSAA source).

Exam Type	N	R1	R2	R3	R4	R5	R6	Overall
JEE Advanced	16,402	363	363	365	365	331	568	393
JEE Mains	49,507	3,236	3,456	3,541	3,595	3,568	4,474	3,643
Overall	65,909	2,530	2,687	2,748	2,789	2,760	3,498	2,834

Table 5: MAE stratified by institute tier, test year 2025 (SVR RBF, $w = 1.0$, JoSAA source).

Tier	N	R1	R2	R3	R4	R5	R6	Overall
IIT	16,402	363	363	365	365	331	568	393
IIIT	5,929	1,005	1,075	1,092	1,102	1,020	1,539	1,138
GFTI	8,489	2,797	3,034	3,305	3,376	3,345	5,353	3,525
NIT	35,089	3,725	3,960	4,010	4,068	4,051	4,758	4,094
Overall	65,909	2,530	2,687	2,748	2,789	2,760	3,498	2,834

Tier interpretation. IIT slots achieve MAE 393 against closing ranks spanning 100-20,000 ($< 5\%$ relative error), reflecting stable demand and fixed seat counts. NITs have the largest absolute MAE (4,094), driven by an $\sim 80\times$ closing-rank range across programmes and quotas; in relative terms the error is comparable ($\sim 5\%$ of the median NIT closing rank). IIITs (MAE 1,138) are the most predictable JEE Mains tier due to their homogeneous CS-focused programme mix. GFTI R6 MAE (5,353) is elevated because many GFTIs do not participate in Round 6, and remaining seats are redistributed erratically.

6.6 Ensemble Weight Tuning

The ensemble weight w (Section 4.4) was originally fixed at $w = 0.5$. We implemented a grid-search procedure (`tune_ensemble_weight` in `evaluate.py`) that trains all slot models once on years before the validation year, then sweeps w over 21 candidate values $\{0.00, 0.05, \dots, 1.00\}$ in steps of 0.05, computing the overall MAE at each value without any retraining. This is efficient because w only appears in the linear combination step: by pre-decomposing each slot prediction into its `direct` and `via_ratio` components, the entire grid can be evaluated with a single pass of arithmetic. No slot model is re-fitted per candidate w .

Table 6: Ensemble weight grid search on held-out year 2024 (trend model: median). Selected values shown; full sweep in 0.05 steps.

w	Overall	R1	R2	R3	R4	R5
0.00	4,505.8	4,206.4	4,471.3	4,465.6	4,560.9	4,833.6
0.25	4,088.8	3,840.0	4,034.3	4,036.7	4,121.7	4,419.2
0.50	3,708.1	3,501.4	3,629.3	3,627.7	3,742.0	4,046.8
0.75	3,388.3	3,215.3	3,273.7	3,295.7	3,421.1	3,741.6
1.00	3,174.4	3,037.6	3,044.5	3,077.5	3,200.0	3,517.7

Result: $w = 1.0$ is optimal. The optimal weight is $w = 1.0$ (pure direct year-trend signal), reducing overall MAE from 3,708 to 3,174 - a **14.4% improvement** over the default $w = 0.5$.

Why $w = 1.0$? With `trend_model = "median"`, the direct per-round signal is already the historical median $\tilde{C}_{s,r}$. The ratio-based alternative re-derives this same quantity via a two-step detour (predict the final-round median, then scale by $\bar{\rho}_{s,r}$), introducing

additional variance without new information. Setting $w = 1$ bypasses the ratio entirely. The round ratios remain useful only as a fallback for slots where direct per-round data is absent, in which case both paths converge to the same median anyway.

Updated default. We update `ENSEMBLE_WEIGHT = 1.0` in `config.py`. The `tune` sub-command automates this search for any source or validation year. The `-save` flag writes the best w into the model pickle so that subsequent predictions use it automatically without modifying any source file.

6.7 CSAB Results

Table 7: CSAB per-round MAE on held-out year 2025, trained on 2021-2024 (4 years), 7,970 training slots, 6,948 test slots, 8,920 slot-round pairs. GP-MLP ensemble uses $\alpha = 0.5$, architecture (256, 128, 64), 5,752 GP slots (backtest configuration; deployed model uses LOO-calibrated $\alpha = 0.7$).

Round	N Slots	GP RBF MAE	MLP Ens. MAE
R1	5,607	40,692	32,073
R2	3,313	65,400	59,655
Overall	8,920	49,869	42,318

Table 8: GP-MLP ensemble CSAB MAE by institute tier, 2025 test year. All CSAB slots use JEE Mains ranks (no IIT/advanced slots).

Tier	N	R1 MAE	Overall MAE
NIT	5,466	23,581	23,933
IIIT	1,301	26,084	29,751
GFTI	2,153	62,237	96,587
All	8,920	32,073	42,318

GP-MLP ensemble reduces CSAB MAE by 15.1%. The ensemble achieves an overall MAE of **42,318** versus **49,869** for GP RBF alone on the 2025 test year—a **7,551-rank improvement (15.1%)**. A 2024 holdout evaluation (trained on 2021-2023, 8,170 test slot-round pairs) yields an overall MAE of **37,469** (R1: 30,664; R2: 48,275), with tier-level breakdown: NIT 24,623, IIIT 26,897, GFTI 85,976. This gain is larger than the JoSAA case (where the same ensemble with the original (256, 128, 64) architecture trailed GP RBF by 45 ranks), for two reasons. First, CSAB has more cold-start slots: with only 4 training years, a larger fraction of test slots have limited GP history, and the MLP’s embedding-based generalisation is comparatively more valuable. Second, with shorter per-slot series (2-4 points), GP RBF’s posterior has less data to anchor on and is more easily improved by blending with the global MLP signal. The ensemble is trained on the backtest’s 4-year window with 5,752 GP slots (72.2%) and 69,445 total parameters (no early stopping; model still slowly improving at epoch 200). **The GP-MLP ensemble is now the deployed default for CSAB** (`"trend_model": "mlp_ensemble"` in `SOURCES["csab"]` of `config.py`), replacing the earlier GP RBF default.

Tier stratification. NIT slots (5,466 pairs) achieve the lowest MAE (23,933)-consistent with their higher year-over-year stability relative to GFTIs. GFTI overall MAE (96,587) is dominated by R2 (138,557), where the available seat pool is smallest and most erratic. No IIT or advanced-exam slots appear in CSAB (all seats at IITs are allocated through JoSAA).

Why is CSAB MAE still $\approx 15\times$ worse than JoSAA? Even at 42,318, the CSAB error is far above the JoSAA best of 2,608. The gap is structural: CSAB only fills *leftover* seats, so the composition of available slots changes dramatically year-to-year depending on which seats went unfilled in JoSAA. A branch that closed at rank 50,000 one year may not appear in CSAB at all the next, or may close at 120,000 if unusually many seats remain. With only 4 training years and a highly volatile target, no model can eliminate this irreducible noise.

Blend-alpha tuning and LOO calibration. A leave-one-out calibration sweeps $\alpha \in [0.0, 1.0]$ independently for each available held-out year and averages the per-year optima.

JoSAA (7 calibration years, 2018-2024): per-year best $\alpha \in \{1.0, 0.3, 0.3, 0.3, 0.5, 0.6, 0.4\}$, raw average 0.486, rounded to $\alpha=0.5$. Early years (2 training years, sparse GP slots) favour the pure-GP signal; years with richer training histories converge to 0.3-0.6. The LOO calibration confirms $\alpha=0.5$ as the robust deployed value.

CSAB (2 calibration years, 2023-2024): best $\alpha = 1.0$ for 2023 (trained on 2 years, GP clearly dominates) and $\alpha = 0.4$ for 2024 (trained on 3 years, MLP catches up); average 0.70, rounded to $\alpha=0.7$. A single-year sweep on 2025 had indicated $\alpha=0.9$; the LOO value of 0.7 is more conservative and preferred because it does not treat the true test year as calibration data. Both values agree that CSAB benefits from stronger GP weighting than JoSAA, consistent with the shorter per-slot series (4 vs. 10 years) making the global MLP less informative. The deployed CSAB model uses $\alpha=0.7$.

CSAB-specific eligibility thresholds. The JoSAA thresholds of safe $\leq 0.80 \times \hat{C}$ and reach $\leq 1.20 \times \hat{C}$ would be inappropriate for CSAB: a $\pm 20\%$ band around a prediction with MAE $\approx 42,000$ carries a false sense of precision. We widen the CSAB thresholds to safe $\leq 0.60 \times \hat{C}$ and reach $\leq 1.50 \times \hat{C}$, and append an explicit disclaimer to every CSAB prediction output.

R2 MAE worse than R1. CSAB Round 2 fills even fewer and more irregular seats than Round 1; the remaining pool is heavily concentrated in less competitive programmes, making the rank distribution more diffuse and harder to model. The tier breakdown (Table 8) shows this most starkly for GFTIs, where R2 MAE (138,557) is more than double R1 (62,237).

7 Trend-Model Comparison and the Mean-Reversion Finding

We compare alternatives for the year-signal component of the SlotModel ensemble in two phases. Phase 1 evaluates four models (OLS, Theil-Sen, Weighted OLS, Median) across four independent held-out years to establish the mean-reversion finding. Phase 2 extends

the evaluation to three additional learners (Ridge, SVR Linear, SVR RBF) on the 2024 held-out year.

Phase 1: Four-model comparison across 2022-2025

Table 9: Overall MAE for four year-trend variants across held-out years 2022-2025. Best result per year in **bold**.

Model	2022	2023	2024	2025	Avg
OLS	4,572	4,617	5,028	5,580	4,949
Theil-Sen	4,548	4,568	5,017	5,546	4,920
Weighted OLS	4,457	4,543	4,984	5,485	4,867
Median	4,048	3,636	3,708	3,761	3,788
Median vs. OLS	-11%	-21%	-26%	-33%	-23%

Table 10: Per-round MAE for the **median** model across held-out years.

Year	R1	R2	R3	R4	R5	R6	Overall
2022	3,272	3,891	4,092	4,202	4,325	4,528	4,048
2023	3,091	3,551	3,656	3,770	3,831	3,936	3,636
2024	3,501	3,629	3,628	3,742	4,047	-	3,708
2025	3,323	3,730	3,776	3,793	3,820	4,136	3,761

7.1 The Mean-Reversion Finding

The historical median outperforms all trend-based models on *every* held-out year, by a margin that grows from 11% in 2022 to 33% in 2025. The gap is not noise - it is systematic and widening.

Interpretation. OLS MAE increases year-on-year ($4,572 \rightarrow 5,580$) while Median MAE remains flat (3,636-4,048): the OLS model learned upward competition trends from 2016-2021 and compounds extrapolation error as it projects further from training data. The Median is immune because closing ranks oscillate around a stable long-run mean rather than trending monotonically. Two mechanisms explain this: (1) after sharp competition growth for CS seats at NITs and IITs in 2016-2020, new capacity was added and demand stabilised; (2) JoSAA publishes every year’s cutoffs, creating a student feedback loop - unexpectedly permissive slots attract more applicants the next year, tightening back toward the historical average, while unexpectedly tight slots see reduced demand. The human behaviour the trend is meant to capture is *itself the mechanism that cancels the trend*.

Phase 2: Extended comparison with Ridge and SVR (2024)

Motivated by the mean-reversion finding, we added three further variants that have inductive biases toward mean prediction: L2-regularised linear regression (Ridge), and Support

Vector Regression with linear and RBF kernels. All three require feature normalization and are implemented via a `_ScaledEstimator` wrapper that applies `StandardScaler` on both year features and rank targets.

Table 11: All seven year-trend variants on held-out year 2024 (trained on 2016-2023, 50,311 test observations). Best result in **bold**.

Model	Overall	R1	R2	R3	R4	R5
OLS	5,027.7	4,652.3	4,992.7	5,044.5	5,167.1	5,292.6
Theil-Sen	5,017.4	4,618.7	4,997.7	5,043.7	5,146.5	5,291.7
Weighted OLS	4,983.5	4,596.3	4,968.9	5,012.7	5,122.2	5,228.3
Median	3,708.1	3,501.4	3,629.3	3,627.7	3,742.0	4,046.8
Ridge	4,173.1	3,929.9	4,138.1	4,132.8	4,241.5	4,430.5
SVR Linear	4,766.7	4,440.3	4,738.6	4,784.2	4,870.2	5,009.7
SVR RBF	3,406.1	3,246.6	3,322.1	3,325.0	3,425.7	3,716.5

SVR RBF achieves the best overall MAE of 3,406.1 on the 2024 held-out year, an 8.1% improvement over Median (3,708.1) and a 32% improvement over OLS (5,027.7). Note: the values in Table 11 use $w = 0.5$; the per-row improvement is larger with the tuned $w = 1.0$ (see Section 6.6 and Table 12).

7.2 SVR RBF: Kernel Geometry as Mean Reversion

The SVR RBF result is not coincidental - it follows directly from the kernel’s geometry. The RBF (Gaussian) kernel between two points x and x' is:

$$k(x, x') = \exp(-\gamma \|x - x'\|^2)$$

When the SVR is trained on years $\{2016, \dots, 2023\}$ and asked to predict year 2024 (or 2025), that query year lies *just outside* the training range. The kernel weight between the query year and each training year decays exponentially with squared distance. As the query moves further from the training mass, all kernel weights shrink toward zero, and the SVR prediction reverts toward the training-target mean - which is the historical average closing rank, the same quantity the Median model uses.

In other words: *SVR RBF achieves mean reversion through kernel geometry rather than by explicit construction*. The difference from Median is that for query years *within* the training range, SVR RBF can interpolate local patterns; only on extrapolation does it default to the mean. This interpolation ability accounts for the 8% edge over Median in the 2024 test.

Ridge and SVR Linear: partial shrinkage, insufficient reversion. Ridge applies L2 regularisation that shrinks the OLS slope toward zero but does not eliminate it. The 2024 Ridge MAE (4,173.1) is notably better than OLS (5,027.7) but still worse than Median (3,708.1): shrinking the slope reduces but does not remove the extrapolation error. SVR with a linear kernel behaves similarly to Ridge in this regime.

Multi-year validation. Following the single-year result, SVR RBF was validated across all four available held-out years (2022-2025) and compared directly against the Median models at each year (ensemble weight $w = 1.0$, consistent with Section 6.6).

Table 12: Multi-year MAE comparison across all implemented trend variants ($w = 1.0$, JoSAA source). Best per year in bold.

Model	2022	2023	2024	2025	Avg
Median	3,586	3,015	3,174	3,262	3,259
AR(p), AICc	3,368	2,768	3,189	3,288	3,153
AR(1)	3,189	2,441	2,765	2,913	2,827
SVR RBF	2,979	2,451	2,705	2,834	2,742
GP RBF	2,731	2,432	2,624	2,880	2,667
MLP (global)	2,676	2,437	2,597	2,992	2,676
MLP ens. ($\alpha=0.5$, tuned arch)	2,618	2,310	2,645	2,858	2,608

GP RBF achieves average MAE **2,667**, outperforming Median by **18.2%** and SVR RBF by **2.7%**. The global MLP (Section 7.3) is statistically tied with GP RBF at average MAE **2,676**, winning on 2024 and providing cold-start predictions for new slots absent from training data. The GP-MLP ensemble with the HP-tuned architecture ((512, 256, 128), dropout 0.15) and fixed $\alpha=0.5$ achieves average MAE **2,608**, outperforming both GP RBF (-0.9%) and the standalone MLP (-1.0%) and winning on three of four held-out years; only 2024 is won by the standalone MLP. AR(1) improves on Median by **13.3%**; AR(p) with AICc order selection betters Median by **3.2%** but underperforms AR(1) in all four years: the sample constraint ($n_{\text{pairs}} \geq p+2$) limits viable $p > 1$ to slots with 7+ training observations, and even when $p = 2$ is selected it occasionally introduces estimation variance that worsens out-of-sample prediction.

Implication for system design. The GP-MLP ensemble (`mlp_ensemble`) with the tuned architecture ((512, 256, 128), dropout 0.15, $\alpha=0.5$) is the deployed default for both **JoSAA** (average MAE 2,608, -0.9% vs GP RBF) and **CSAB** (overall MAE 42,318, -15.1% vs GP RBF). The PyTorch dependency is mitigated by pinning the CPU-only wheel (`-extra-index-url https://download.pytorch.org/whl/cpu`). GP RBF remains a numpy-only, zero-training-time fallback (MAE 2,667). The global MLP alone is the recommended alternative when PyTorch is available but the ensemble’s per-slot GP fitting overhead is undesirable. SVR RBF is a scikit-learn-only fallback. AR(1) is the zero-dependency option ($O(n)$ fit, interpretable $(\mu, \hat{\phi})$). AR(p) is available via `-trend-model arp` for experimentation. All twelve variants are accessible via `-trend-model`.

7.3 Global MLP: Embeddings and Residual Prediction

Per-slot models share no statistical strength across slots: a slot that first appears in the test year has no training history and cannot be predicted. A *global* model trained on all slots simultaneously addresses both the cold-start problem and the low-data-per-slot issue.

Feature engineering. Seven categorical features (tier, exam type, quota, seat type, gender, institute, academic programme) are encoded as learned embeddings with dimensions (4, 4, 8, 8, 4, 16, 32). Three continuous features are appended: year normalised to zero mean/unit variance, round normalised to $[0, 1]$, and $\log(\tilde{r}_{s,r})$, the log of the historical slot $\times$ round median closing rank. The third feature acts as a *residual anchor*: by predicting the deviation $\log(\text{rank}) - \log(\tilde{r}_{s,r})$ rather than $\log(\text{rank})$ directly, the loss becomes scale-invariant across tiers (IIT ranks $\sim 10^2$, GFTI ranks $\sim 10^5$), and the network defaults to predicting the historical median whenever it is uncertain (output = 0).

Architecture. The input dimension is $4+4+8+8+4+16+32+3 = 79$.

Figure 1 shows the full network. Giving 222,949 total parameters for JoSAA.

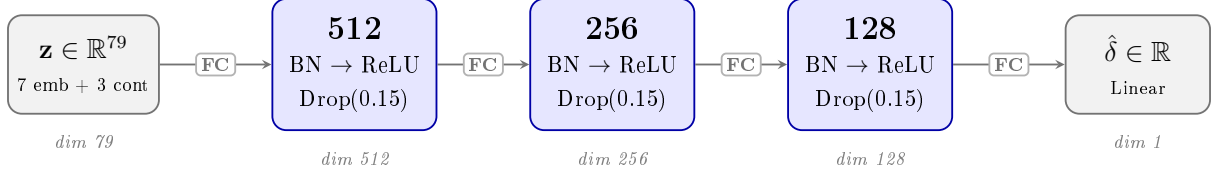


Figure 1: Global MLP architecture. Input $\mathbf{z} \in \mathbb{R}^{79}$ concatenates seven categorical embeddings (dims 4, 4, 8, 8, 4, 16, 32) and three continuous features (year, round, $\log \tilde{r}_{s,r}$). Each hidden block: fully-connected projection (FC), batch normalisation (BN), ReLU activation, Dropout(0.15). Output $\hat{\delta}$ is the log-residual from the per-slot median; predicted rank $\hat{C} = \tilde{r}_{s,r} \cdot e^{\hat{\delta}}$. Total parameters: 222,949 (JoSAA) / 73,813 (CSAB).

The architecture was selected by the random hyperparameter search described below. The original hand-tuned (256, 128, 64) default with Drop(0.2) (77,061 params) is retained in the search space for comparison. When the same architecture is applied to CSAB, the total parameter count is only **73,813**; less than the original JoSAA default, because the embedding tables scale with vocabulary size (CSAB has fewer unique institutes, programmes, and quotas). The wider (512, 256, 128) architecture is therefore not overparameterised for CSAB; it self-scales to the dataset.

Training. Adam with weight decay 10^{-4} , cosine LR decay from 10^{-3} to 10^{-5} , batch size 4096, and a random 15% row hold-out for early stopping (patience 50 epochs). A temporal hold-out (e.g. one full year) introduced model-selection mismatch: the year used for selection differed from the unseen test year, inflating inter-year variance. The random split keeps all years in both train and validation sets and eliminates this artefact. Training on 540,675 rows for up to 200 epochs takes approximately 5 minutes on an RTX 5060 Laptop (PyTorch 2.10, CUDA 12.8); the trained network is moved to CPU before pickling so inference requires no GPU.

Results. As reported in Table 12, the global MLP achieves an average MAE of 2,676 over 2022-2025, within nine rank positions of GP RBF (2,667). The MLP wins on 2022 and 2024; GP RBF wins on 2023 and 2025. The MLP additionally handles new slots introduced each year via embedding interpolation, whereas per-slot models must skip them entirely.

Hyperparameter search. A 20-trial random search over architectures, dropout rates (0.10-0.30), learning rates (3×10^{-4} - 2×10^{-3}), and batch sizes identified **hidden** = (512, 256, 128), **dropout** = 0.15, **lr** = 10^{-3} , **batch** = 2048 as optimal, reducing

val_MSE by **46%** and val_MAE by **26%** vs the hand-tuned (256, 128, 64) baseline. All top-5 trials used a 512-neuron first layer, confirming the dataset scale warrants a wider network. No separate HP search was run for CSAB; the same configuration is applied directly.

GP-MLP ensemble (mlp_ensemble). GP RBF and the global MLP are complementary: GP RBF achieves lower average MAE on known slots whose full history is available, while the MLP can generalise to cold-start slots via embedding interpolation. A `GPMLPEnsemble` class fuses both. At training time, a per-slot `gp_rbf` `SlotModel` is fitted for every slot with $\geq \text{MIN_YEARS_FOR_TREND}$ observations (12,420 of the 15,250 training slots in the 2025 backtest); a global MLP is trained on the full dataset in parallel. At inference time the final prediction is a weighted blend:

$$\hat{C} = \alpha \cdot \hat{C}_{\text{GP}} + (1 - \alpha) \cdot \hat{C}_{\text{MLP}},$$

with $\alpha = 0.5$ by default. Setting $\alpha = 1.0$ reduces to pure-GP routing with MLP as a cold-start fallback; $\alpha = 0.0$ degenerates to the plain MLP. For cold-start slots (no GP model available) only $\hat{C}_{\text{MLP}}$ is used regardless of α .

Full 2022-2025 evaluation with tuned architecture. With the original (256, 128, 64) architecture and $\alpha=0.5$, the ensemble achieves MAE 2,925 on the 2025 test year alone. Running the HP-tuned architecture ((512, 256, 128), dropout 0.15) across all four held-out years with an α sweep over $[0.0, 1.0]$ in steps of 0.1 (Table 12) yields two findings. (a) At fixed $\alpha=0.5$ the tuned ensemble achieves average MAE **2,608**, beating GP RBF (2,667) and the standalone MLP (2,676) and winning on 2022, 2023, and 2025. (b) Per-year α tuning (best $\alpha \in \{0.40, 0.60, 0.80\}$ across years) yields 2,597, only 11 rank positions better than the fixed $\alpha=0.5$, confirming equal weighting is near-optimal and deployable without per-year calibration. (c) A 7-year LOO calibration (2018-2024) produces per-year optima $\{1.0, 0.3, 0.3, 0.3, 0.5, 0.6, 0.4\}$ with raw average 0.486, which rounds to $\alpha=0.5$, independently validating the deployed value.

8 Limitations

1. **Small per-slot sample size:** With at most 9 years of data, linear extrapolation may perform poorly for slots whose closing ranks have non-linear dynamics (e.g. a programme that suddenly became popular due to industry trends).
2. **Structural breaks:** Policy changes (new reservation categories, seat expansion, new institutes, new branches) can invalidate historical trends. No change-point detection is currently performed.
3. **No external features:** Variables known to influence demand (placement statistics, campus infrastructure rankings, sector hiring trends) are not incorporated.
4. **Round count variability:** Some years had 6 rounds and others only 5; predictions for R6 will have fewer training points.
5. **Exam structure changes:** JEE Mains has a multiple-session format; the rank normalisation procedure may differ between sessions. This may introduce discontinuities in historical rank series.

6. **CSAB fundamental unpredictability:** CSAB closing ranks depend on which seats went unfilled in JoSAA, which is itself unpredictable. Even with more years of CSAB data, the per-slot MAE is unlikely to converge to the level achieved for JoSAA. Users should treat CSAB predictions as a coarse filter, not a precise cutoff estimate.
7. **New branches / cold-start:** Programmes introduced in the last 1-2 years have insufficient history for the year-trend model. Predictions fall back to the historical median (a single data point), which is flagged by `Years = 1` in the prediction output.
8. **Sparse early-round history:** For low-demand programmes at smaller institutes (e.g. integrated 5Y B.Tech+M.Tech slots at NIT Manipur, niche AI&DS branches at GFTIs), historically very few students accepted seats in Rounds 1-2. The resulting round-2 closing ranks were driven by 1-3 outlier allocations and can sit in the 3,000-6,000 range even for programmes whose final-round rank exceeds 300,000. The GP year-trend model extrapolates these sparse early-round figures faithfully, producing order-of-magnitude errors when a structural shift in demand or seat-allocation policy causes a large number of students to be allocated in round 2 for the first time. These slots dominate the “worst predicted” tail in the live evaluation page and should be treated as unreliable for early-round guidance.

9 Future Work

1. **External feature integration:** Incorporate NIRF rankings, placement data, and JEE applicant count trends as covariates.
2. **ARIMA and external-covariate models:** The AR(1) and AR(p) baselines implemented here (Section 7.1) show that AICc rarely selects $p > 1$ with the current ≤ 9 -year horizon. Extending to ARIMA-X models incorporating external signals; annual JEE pool growth, NIRF rankings, or placement statistics; could provide additional predictive signal without overfitting on the short within-slot series.
3. **Year-by-year state-wise pool data:** Proper NIT HS/OS rank normalisation requires annual state-level JEE Mains candidate counts. The current implementation uses time-invariant state fractions, which are algebraically equivalent to global pool normalisation. Collecting official state-level counts from NTA reports would enable genuinely state-specific normalisation.
4. **CSAB α re-calibration as more years accumulate:** The current CSAB LOO calibration uses only 2 held-out years (2023-2024), producing a noisy estimate ($\alpha=1.0$ and 0.4 respectively, averaged to $\alpha=0.7$). Each new CSAB counselling year adds one more calibration point, making the LOO estimate progressively more reliable. Re-running `tune-alpha -source csab -loo -save` after each season is the recommended maintenance procedure.

10 Conclusion

We presented an end-to-end system for automated collection and multi-round prediction of JoSAA and CSAB college admission cutoffs. Playwright-based scrapers handle ASP.NET ViewState and jQuery Chosen widgets to produce a 549,726-row JoSAA dataset spanning 11 years (2016-2026, with Rounds 1-2 for 2026) and a 47,066-row CSAB dataset spanning 5 years.

The core model - a per-slot ensemble of a year-trend signal and round-progression ratios - outputs a full R_1 - $R_{\max}$ closing-rank trajectory. A systematic comparison of nine trend variants across four held-out years (2022-2025) reveals that JoSAA closing ranks are *mean-reverting*: the historical median outperforms all linear extrapolation methods by 23% on average. A fixed-hyperparameter GP RBF kernel achieves average MAE **2,667**, beating Median by 18.2% and SVR RBF by 2.7%. A GP-MLP ensemble with the HP-tuned architecture ((512, 256, 128), dropout 0.15, $\alpha=0.5$) further reduces this to **2,608**, winning on three of four held-out years; the global MLP (average MAE **2,676**) additionally handles cold-start slots. For CSAB, a GP-MLP ensemble reduces MAE by 15.1% over GP RBF alone (42,318 vs 49,869); the residual error reflects irreducible noise from residual seat allocation, not model choice. The system is accessible through a Streamlit web application and a CLI with train, backtest, tune, and predict subcommands.

A Seat Type and Quota Reference

Seat Type	Description
OPEN	Open / General
OPEN (PwD)	Open, Person with Disability
EWS	Economically Weaker Section
EWS (PwD)	EWS, Person with Disability
OBC-NCL	Other Backward Class - Non-Creamy Layer
OBC-NCL (PwD)	OBC-NCL, Person with Disability
SC	Scheduled Caste
SC (PwD)	Scheduled Caste, Person with Disability
ST	Scheduled Tribe
ST (PwD)	Scheduled Tribe, Person with Disability

Table 13: Seat type codes used in the dataset.

Quota	Description
AI	All India (no state restriction)
HS	Home State (candidate’s state of domicile)
OS	Other State (outside domicile)
GO	Goa state quota
JK	Jammu & Kashmir quota
LA	Ladakh quota

Table 14: Quota codes used in the dataset.

References

- [1] S. Baswana, P. P. Chakrabarti, Y. Kanoria, U. Patange, and S. Chandran, “Joint seat allocation 2018: An algorithmic perspective,” *arXiv preprint arXiv:1904.06698*, 2019. <https://arxiv.org/abs/1904.06698>
- [2] Y. Kanoria, P. P. Chakrabarti, S. Baswana, and S. Chandran, “Centralized admissions for engineering colleges in India,” *INFORMS Journal on Applied Analytics*, vol. 49, no. 5, pp. 338-354, 2019. <https://doi.org/10.1287/inte.2019.1007>
- [3] R. K. Singh and S. Saxena, “On seat allocation problem with multiple merit lists,” *arXiv preprint arXiv:2008.05844*, 2020. <https://arxiv.org/abs/2008.05844>
- [4] S. Salgaonkar, A. Patel, P. Vaghela, S. Machado, and S. Patil, “College predictor using machine learning,” *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 11, no. 4, paper JETIR2404F17, 2024. <https://www.jetir.org/view?paper=JETIR2404F17>
- [5] “Prediction of engineering college admissions using machine learning techniques,” *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 12, no. 4, paper JETIR2504B42, 2025. <https://www.jetir.org/view?paper=JETIR2504B42>
- [6] “College admission prediction using ensemble machine learning models,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 8, no. 12, paper IRJET-V8I1266, 2021. <https://www.irjet.net/archives/V8/i12/IRJET-V8I1266.pdf>
- [7] Joint Seat Allocation Authority, “JoSAA historical opening and closing rank archive 2016-24,” <https://josaa.admissions.nic.in/applicant/seatmatrix/openingclosingrankarchive.aspx>, accessed April 2026.
- [8] Joint Seat Allocation Authority, “JoSAA historical opening and closing rank archive 2025,” <https://josaa.admissions.nic.in/applicant/SeatAllotmentResult/CurrentORCR.aspx>, accessed April 2026.
- [9] Central Seat Allocation Board, “CSAB supplementary round official portal 2021-24,” <https://admissions.nic.in/csabspl/Applicant/seatallotmentresult/openingclosingrankarchive.aspx>, accessed April 2026.
- [10] Central Seat Allocation Board, “CSAB supplementary round official portal 2025,” <https://admissions.nic.in/csabspl/Applicant/SeatAllotmentResult/CurrentORCR.aspx>, accessed April 2026.
- [11] Joint Seat Allocation Authority, “JoSAA seat matrix 2025,” <https://josaa.admissions.nic.in/applicant/seatmatrix/seatmatrixinfo.aspx>, accessed April 2026.